



Clases anónimas, lambdas y adaptadores: un receptor por evento

El problema

La caja de la clase pasada quedó repartida en tres clases. Caja sabe sumar pagos y no importa nada de `javax.swing`. VistaCaja arma la ventana y sabe mostrar. ControladorCaja implementa `ActionListener`, se registra en el botón, y en `actionPerformed()` traduce entre las otras dos.

El local cobra en efectivo y con tarjeta de débito, y son dos pagos distintos, porque el banco se queda con una comisión por cada débito. Así que el cajero pide un segundo botón.



La ventana con dos botones, después de registrar 1500 en efectivo y 800 con débito.

La vista lo agrega sin problema: el botón “Registrar” pasa a decir “Efectivo”, aparece un `JButton` más, y `getBotonDebito()` para exponerlo. El modelo ya tenía `PagoDebito` desde la clase 5. Y el controlador, que hasta ahora atendía un solo botón, se registra en los dos y adentro pregunta cuál fue la fuente:



```
public ControladorCaja(VistaCaja vista, Caja caja) {
    // ...
    this.vista.getBotonEfectivo().addActionListener(this);
    this.vista.getBotonDebito().addActionListener(this);
}

@Override
public void actionPerformed(ActionEvent evento) {
    try {
        double monto = Double.parseDouble(this.vista.getTextoMonto());
        if (evento.getSource() == this.vista.getBotonDebito()) {
            this.caja.registrar(new PagoDebito(monto));
        } else {
            this.caja.registrar(new PagoEfectivo(monto));
        }
        this.vista.mostrarTotal(this.caja.total());
    } catch (NumberFormatException ex) {
        this.vista.mostrarError("Ingrese un numero");
    } catch (IllegalArgumentException ex) {
        this.vista.mostrarError(ex.getMessage());
    }
}
```

Es lo mismo que terminó haciendo la actividad de la clase pasada con los euros, y funciona. El costo aparece con el tercer botón. El local empieza a cobrar con código QR, y para agregarlo hay que abrir `actionPerformed()`, que ya funcionaba, y agregarle un caso. Con cinco medios de pago son cinco casos en el mismo método. Es la cadena de `if` de la clase 7, y el principio que falla es Open/Closed: agregar algo obliga a modificar lo que ya estaba.

Hay un segundo problema, más escondido. El cajero también pide que la tecla Enter, escrita en el campo del monto, registre en efectivo, para no soltar el teclado. Eso no es un `ActionEvent` de un botón. Es un evento de teclado, con otra interfaz y otro método, y no entra en ese `actionPerformed()` por más `if` que se le agreguen.

Lo que falta es un mecanismo para que cada evento tenga su propio receptor, sin que cada receptor cueste un archivo. Y una regla que ordena esta clase: un receptor por evento, escrito donde se registra.

Primer intento: una clase por botón

El receptor de un botón es cualquier objeto que implemente `ActionListener`. Nada obliga a que sea el controlador. Se puede escribir una clase por botón:



```
public class ReceptorEfectivo implements ActionListener {

    private final VistaCaja vista;
    private final Caja caja;

    public ReceptorEfectivo(VistaCaja vista, Caja caja) {
        this.vista = vista;
        this.caja = caja;
    }

    @Override
    public void actionPerformed(ActionEvent evento) {
        try {
            double monto = Double.parseDouble(this.vista.getTextoMonto());
            this.caja.registrar(new PagoEfectivo(monto));
            this.vista.mostrarTotal(this.caja.total());
        } catch (NumberFormatException ex) {
            this.vista.mostrarError("Ingrese un numero");
        } catch (IllegalArgumentException ex) {
            this.vista.mostrarError(ex.getMessage());
        }
    }
}
```

ReceptorDebito es igual con PagoDebito. El controlador deja de implementar ActionListener y su constructor se reduce a crear y registrar:

```
vista.getBotonEfectivo().addActionListener(new ReceptorEfectivo(vista, caja));
vista.getBotonDebito().addActionListener(new ReceptorDebito(vista, caja));
```

El if desapareció. Agregar el QR es escribir ReceptorQR y una línea en el constructor, sin tocar los otros dos. Open/Closed se cumple.

El costo se ve en la carpeta. Cada botón es un archivo con su clase, sus dos atributos y su constructor, y los dos receptores repiten los mismos dos atributos para llegar a la misma vista y a la misma caja. Cada receptor tiene unas veinte líneas, de las cuales una es la que importa. Y para saber qué hace el botón de efectivo hay que salir del controlador y abrir otro archivo.

Una clase por botón resuelve el if y crea otro problema: un archivo por línea que importa.

Clases anónimas

Java permite declarar una clase y crear su único objeto en el mismo lugar, sin darle nombre. A eso se lo llama clase anónima, y la sintaxis es `new` seguido de la interfaz que implementa y de un bloque con los métodos:



```
public ControladorCaja(VistaCaja vista, Caja caja) {  
    // ...  
    this.vista.getBotonEfectivo().addActionListener(new ActionListener() {  
        @Override  
        public void actionPerformed(ActionEvent evento) {  
            registrarEfectivo();  
        }  
    });  
}  
  
private void registrarEfectivo() {  
    // Lo que hacia ReceptorEfectivo.actionPerformed()  
}
```

`new ActionListener() { ... }` no crea un `ActionListener`, porque una interfaz no se puede instanciar. Crea un objeto de una clase nueva, sin nombre, que implementa `ActionListener` con el `actionPerformed()` escrito entre las llaves. Ese objeto se pasa directamente a `addActionListener()`. La clase existe, el compilador le pone un nombre interno, pero desde el código no hace falta nombrarla, porque no se la vuelve a usar.

Lo que cambia respecto de `ReceptorEfectivo` es dónde está escrita. Al estar adentro del constructor del controlador, la clase anónima puede usar lo que el controlador tiene: sus atributos y sus métodos, como `registrarEfectivo()`, que es privado y aun así se puede llamar. Por eso no necesita ni constructor ni atributos propios. Las veinte líneas de `ReceptorEfectivo` se convierten en seis, y están al lado del botón al que atienden.

Hay un detalle que confunde la primera vez. Adentro de la clase anónima, `this` es el objeto anónimo, no el controlador. Escribir `this.registrarEfectivo()` ahí no compila, porque el `ActionListener` anónimo no tiene ese método. Se llama sin `this`, y Java busca el método primero en la clase anónima y después en la clase que la contiene.

Una clase anónima es una clase declarada y creada en el mismo lugar, y ese lugar es donde se registra.

Lambdas

De las seis líneas de la clase anónima, cinco son sintaxis. `new ActionListener()`, el `@Override`, la firma de `actionPerformed()`, las llaves. El compilador ya sabe que `addActionListener()` recibe un `ActionListener`, y ya sabe que `ActionListener` tiene un solo método. Lo único que no puede adivinar es el cuerpo.

Una expresión lambda es exactamente eso, el cuerpo y nada más:

```
this.vista.getBotonEfectivo().addActionListener(evento -> this.registrarEfectivo());  
this.vista.getBotonDebito().addActionListener(evento -> this.registrarDebito());
```

Tiene tres partes. A la izquierda de la flecha van los parámetros, que son los de `actionPerformed()`, un `ActionEvent`; el tipo no se escribe porque el compilador lo sabe, y el nombre lo elige quien escribe. La flecha `->` separa. A la derecha va el cuerpo: una expresión sola, como acá, o un bloque entre llaves si hace falta más de una línea. Cada lambda es un objeto que implementa `ActionListener`, igual que la clase anónima, y Swing lo llama con cada



click. Adentro de una lambda, además, `this` sigue siendo el controlador, así que `this.registrarEfectivo()` compila.

Una lambda solo funciona cuando la interfaz tiene exactamente un método abstracto. Si tuviera dos, el compilador no sabría cuál de los dos es el cuerpo que se le está dando. A esas interfaces de un solo método se las llama interfaces funcionales, y `ActionListener` es una. Notificador, de la clase 8, también: tiene solo `enviar(String mensaje)`, y `new Caja(mensaje -> { System.out.println(mensaje); return true; })` es una caja con un notificador que imprime, sin escribir ninguna clase.

Con las dos lambdas, el controlador queda así:

```
public class ControladorCaja {  
  
    private final VistaCaja vista;  
    private final Caja caja;  
  
    public ControladorCaja(VistaCaja vista, Caja caja) {  
        if (vista == null || caja == null) {  
            throw new IllegalArgumentException("La vista y la caja son obligatorias");  
        }  
        this.vista = vista;  
        this.caja = caja;  
        this.vista.getBotonEfectivo().addActionListener(evento -> this.registrarEfectivo());  
        this.vista.getBotonDebito().addActionListener(evento -> this.registrarDebito());  
    }  
  
    private void registrarEfectivo() {  
        try {  
            this.registrar(new PagoEfectivo(this.leerMonto()));  
        } catch (NumberFormatException ex) {  
            this.vista.mostrarError("Ingrese un numero");  
        } catch (IllegalArgumentException ex) {  
            this.vista.mostrarError(ex.getMessage());  
        }  
    }  
  
    // registrarDebito() es igual, con PagoDebito  
  
    private double leerMonto() {  
        return Double.parseDouble(this.vista.getTextoMonto());  
    }  
  
    private void registrar(Pago pago) {  
        this.caja.registrar(pago);  
        this.vista.mostrarTotal(this.caja.total());  
    }  
}
```

Ya no dice `implements ActionListener` y no tiene `actionPerformed()`. El controlador dejó de ser el receptor. Tiene un receptor por botón, cada uno en una línea, y un método privado por cosa que puede pasar. Agregar el QR es una línea en el constructor y un método nuevo, y ninguna de las líneas que ya estaban se modifica.



Antes y después. A la izquierda, un receptor para todas las fuentes. A la derecha, un receptor por fuente, escrito donde se registra.

Una lambda es el cuerpo del único método de una interfaz. El compilador pone el resto.

Interfaces con varios métodos

Queda el pedido del Enter. Para enterarse de lo que pasa en el teclado, un componente acepta receptores que implementen `KeyListener`, y esa interfaz tiene tres métodos:

```
public interface KeyListener {  
    void keyTyped(KeyEvent evento);    // se escribió un caracter  
    void keyPressed(KeyEvent evento);  // se apreto una tecla, cualquiera  
    void keyReleased(KeyEvent evento); // se soltó  
}
```

Para el Enter alcanza con `keyPressed()`. Pero una clase que implementa `KeyListener` tiene que escribir los tres, aunque dos queden vacíos, porque así funciona una interfaz desde la clase 6. Y una lambda no sirve, porque son tres métodos y el compilador no tendría forma de saber a cuál corresponde el cuerpo.

Para esto Java trae `KeyAdapter`, una clase que implementa `KeyListener` con los tres métodos vacíos. No hace nada, y esa es la idea: se hereda de ella y se sobrescribe solo el método que interesa. Con una clase anónima, porque tampoco hace falta nombrarla:

```
this.vista.getCampoMonto().addKeyListener(new KeyAdapter() {  
    @Override  
    public void keyPressed(KeyEvent evento) {  
        if (evento.getKeyCode() == KeyEvent.VK_ENTER) {  
            registrarEfectivo();  
        }  
    }  
});
```

Es la misma sintaxis de la clase anónima de antes, con una diferencia: `new KeyAdapter()` hereda de una clase en vez de implementar una interfaz. El objeto `KeyEvent` dice qué tecla fue, con `getKeyCode()`, y `KeyEvent.VK_ENTER` es la constante para el Enter; hay una por tecla, `VK_UP`,



VK_ESCAPE, VK_A. Y como es una clase anónima, registrarEfectivo() va sin this. El mismo método que atiende el botón atiende la tecla.

Para el mouse es igual. MouseListener tiene cinco métodos, click, presionar, soltar, entrar y salir del componente, y MouseAdapter los trae vacíos. Se sobrescribe mouseClicked() y el MouseEvent dice, entre otras cosas, cuántos clicks seguidos fueron, con getClickCount().

Hay un cambio en la vista que es importante notar. Hasta hoy exponía los botones porque eran las fuentes de eventos, y el campo quedaba adentro. Ahora el campo también es fuente de eventos, así que aparece getCampoMonto(). El criterio es el mismo: la vista expone lo que el controlador necesita para registrarse, y nada más.

Con un solo método, lambda. Con varios, una clase anónima sobre el adaptador, que trae los demás vacíos.

Una lambda que no es de un botón

La clase pasada quedó dicho que Swing arranca un hilo propio que se queda esperando eventos, y que actionPerformed() corre en ese hilo. Swing está pensado para que todo lo que toca componentes corra ahí, y el main corre en otro. En los programas del curso la ventana se armó y se mostró directo desde main y no pasó nada, porque son programas chicos y no hay dos hilos tocando lo mismo al mismo tiempo. La forma recomendada es pedirle a Swing que arme la ventana desde su propio hilo:

```
public static void main(String[] args) {  
    SwingUtilities.invokeLater(() -> {  
        Caja caja = new Caja(new ServicioCorreo("smtp.lapampa.com", 587));  
        VistaCaja vista = new VistaCaja();  
        ControladorCaja controlador = new ControladorCaja(vista, caja);  
        vista.mostrar();  
    });  
}
```

invokeLater() recibe una tarea y la pone en la cola del hilo de eventos, para que se ejecute apenas pueda. La tarea es un Runnable, una interfaz con un solo método, run(), sin parámetros. Por eso la lambda arranca con () vacíos y el cuerpo va entre llaves. Es la misma sintaxis que en los botones, con una interfaz que no tiene nada que ver con Swing, y es la señal de que las lambdas no son un truco de los botones: sirven cada vez que un método quiere recibir código para ejecutar después.

invokeLater() recibe qué hacer, no cuándo. Lo mismo que un botón.

Qué se ganó

La ventana se ve igual que al principio, con un botón más y el Enter funcionando. El modelo no cambió, salvo por PagoDebito, que ya existía. La vista expone una fuente más. El controlador es el que cambió de forma: ya no implementa nada, tiene un receptor por evento escrito en la línea donde se registra, y los métodos privados dicen qué puede pasar en la caja. Para saber qué hace un botón se lee su línea, y para agregar uno no se abre nada de lo que ya estaba.



Y las tres formas, la clase con nombre, la clase anónima y la lambda, son el mismo objeto visto desde Swing. Las tres implementan `ActionListener`, las tres se registran con `addActionListener()`, y a las tres las llama Swing con cada click. Lo que cambia es cuánto hay que escribir y dónde queda escrito.

Bibliografía sugerida para profundizar

Deitel, P., & Deitel, H. (2017). *Java How to Program, Early Objects*.

- Capítulo 26, “Swing GUI Components: Part 1”, las secciones sobre clases internas anónimas, eventos de teclado y de mouse, y clases adaptadoras. Capítulo 17, “Lambdas and Streams”, para las lambdas y las interfaces funcionales, más allá de los botones.

Aguilar, L. J. (2011). *Programación en Java: algoritmos, programación orientada a objetos e interfaz gráfica de usuario*. McGraw-Hill.

- Capítulo 21, “Gráficos II. Componentes y eventos”, para los eventos de teclado y de mouse y las clases adaptadoras. Es anterior a las lambdas.

Belmonte Fernández, O. (2012). *Desarrollo de proyectos informáticos con tecnología Java*. Universitat Jaume I.

- Capítulo 11, “Interfaces gráficas de usuario”, secciones 11.4 y 11.5, para escuchadores y adaptadores. Gratuito y breve.

Oracle. *The Java Tutorials*, “Anonymous Classes” y “Lambda Expressions”, en la sección “Classes and Objects”.

- Es la referencia oficial, gratuita, y tiene el detalle de qué puede y qué no puede usar una clase anónima de la clase que la contiene.